

Lab Work for Week 4: URL Routing and Static Files

Objective

By the end of this lab, students will:

1. Understand how to define URL patterns for routing.
 2. Add static files (CSS, images) to the project.
 3. Serve and use static files in templates.
-

Step-by-Step Lab Tasks

1. Organize URLs for the courses App

1. **Task:** Update the `urls.py` file in the courses app.
 - Add named URL patterns for better URL management:

```
from django.urls import path
```

```
from . import views
```

```
urlpatterns = [
    path('', views.course_list, name = 'course_list'),
    path('< int: course_id >/', views.course_detail, name = 'course_detail'),
]
```

2. **Task:** Include app-level URLs in the project's main `urls.py`.
 - Open `EduLearn/urls.py` and ensure it includes the courses URLs:

```
from django.contrib import admin
```

```
from django.urls import path, include
```

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('courses.urls')),
]
```

3. **Task:** Test navigation using named URL patterns in templates.
 - Modify the `course_list.html` template:

```

<ul>
  {% for course in courses %}
    <li>
      <a href = "{% url 'course_detail' course.id %}" > {{ course.title }} </a>
      <p> {{ course.description }} </p>
    </li>
  {% endfor %}
</ul>

```

2. Add Static Files

1. Task: Create a static directory.

- Inside the courses app, create a folder named static.
- Create a subfolder courses for organizing app-specific static files:

```

courses/
  static/
    courses/
      style.css
      logo.png

```

2. Task: Add CSS for styling the course pages.

- Create style.css in courses/static/courses/:

```

body {
  font-family: Arial, sans-serif;
  background-color: #f9f9f9;
  margin: 0;
  padding: 0;
}

```

```

h1, h2 {
  color: #333;
}

```

```

ul {

```

```
list – style – type: none;  
padding: 0;  
}
```

```
li {  
background: #fff;  
margin: 10px 0;  
padding: 10px;  
border: 1px solid #ddd;  
border – radius: 5px;  
}
```

```
a {  
text – decoration: none;  
color: #007BFF;  
}
```

```
a: hover {  
text – decoration: underline;  
}
```

3. Link Static Files in Templates

1. Task: Load and apply the CSS file in course_list.html and course_detail.html.

- Add the following at the top of both templates:

```
{% load static %}
```

```
< link rel = "stylesheet" type = "text/css" href = "{% static 'courses/style.css' %}"  
>
```

2. Task: Display the logo on all pages.

- Add the following to both templates:

```
< img src = "{% static 'courses/logo.png' %}" alt = "EduLearn Logo" width = "100"  
>
```

4. Test Static File Integration

1. Task: Run the development server.

python manage.py runserver

2. Task: Visit the following pages:
 - <http://127.0.0.1:8000/> - Verify that the course list page is styled using the CSS file and displays the logo.
 - http://127.0.0.1:8000/<course_id>/ - Verify that the course detail page uses the same styling and logo.
 3. Task: Inspect the browser console or network tab to confirm that static files are being loaded correctly.
-

Lab Deliverables

1. Code for:
 - Updated `urls.py` files (both app-level and project-level).
 - CSS file (`style.css`).
 - Templates with static file integration (`course_list.html` and `course_detail.html`).
 2. Screenshots of:
 - The styled course list page.
 - The styled course detail page.
-

Additional Questions for Students

1. What is the purpose of `{% load static %}` in Django templates?
2. Why should static files be organized within app-specific subfolders?
3. How does Django locate and serve static files during development?

Extension of Lab Work 4: Django Views, Templates, and Bootstrap Integration

Objective

By the end of this lab, students will:

1. Understand Django views and templates.
 2. Use Bootstrap to style the web pages.
 3. Render dynamic content with Django templates.
-

Step-by-Step Lab Tasks

1. Install Bootstrap in Django

1. **Task:** Add Bootstrap via CDN in Django templates.

- Modify the base template base.html:

```
<!DOCTYPE html >
<html lang = "en" >
<head >
  <meta charset = "UTF - 8" >
  <meta name = "viewport" content = "width = device - width, initial - scale = 1" >
  <title > {% block title %}EduLearn{% endblock %} </title >
  <!-- Bootstrap CSS -->
  <link href = "https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css
    /bootstrap.min.css" rel = "stylesheet" >
</head >
<body class = "bg - light" >
  <nav class = "navbar navbar - expand - lg navbar - dark bg - primary" >
    <div class = "container" >
      <a class = "navbar - brand" href = "/" > EduLearn </a >
      <button class = "navbar - toggler" type = "button" data - bs - toggle
        = "collapse" data - bs - target = "#navbarNav" >
        <span class = "navbar - toggler - icon" ></span >
      </button >
      <div class = "collapse navbar - collapse" id = "navbarNav" >
```

```

< ul class = "navbar - nav" >
    < li class = "nav - item" >< a class = "nav - link" href = "/" > Courses </a >
        </li >
    < li class = "nav - item" >< a class = "nav - link" href = "/enroll/" > Enroll
        </a ></li >
</ul >
</div >
</div >
</nav >
< div class = "container mt - 4" >
    {% block content %}{% endblock %}
</div >
< ! -- Bootstrap JS -- >
< script src = "https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js
    /bootstrap.bundle.min.js" ></script >
</body >
</html >

```

2. **Why This Matters:** This creates a consistent design structure and includes Bootstrap styling.
-

2. Create Views for Course Listing and Details

1. **Task:** Define Django views to display courses.

- Open views.py and add:

```
from django.shortcuts import render, get_object_or_404
```

```
from .models import Course
```

```
def course_list(request):
```

```
    courses = Course.objects.all()
```

```
    return render(request, 'courses/course_list.html', {'courses': courses})
```

```
def course_detail(request, course_id):
```

```
    course = get_object_or_404(Course, id = course_id)
```

```
    lessons = course.lessons.all()
```

```
return render(request, 'courses
                /course_detail.html', {'course': course, 'lessons': lessons})
```

2. **Why This Matters:** Django views process data and pass it to templates.
-

3. Update URLs for the Views

1. **Task:** Define URL patterns in urls.py:

```
from django.urls import path
```

```
from . import views
```

```
urlpatterns = [
    path('', views.course_list, name = 'course_list'),
    path('<int:course_id>/', views.course_detail, name = 'course_detail'),
]
```

4. Create Templates with Bootstrap

1. **Task:** Create course_list.html in templates/courses/.

```
{% extends "courses/base.html" %}
```

```
{% block title %}Courses{% endblock %}
```

```
{% block content %}
```

```
<div class = "container" >
```

```
<h1 class = "text - center" > Available Courses </h1 >
```

```
<div class = "row" >
```

```
{% for course in courses %}
```

```
<div class = "col - md - 4" >
```

```
<div class = "card mb - 4" >
```

```
<div class = "card - body" >
```

```
<h5 class = "card - title" > {{ course.title }} </h5 >
```

```
<p class = "card - text" > {{ course.description }} </p >
```

```
<a href = "{{ course.id }}" class = "btn btn - primary" > View Details </a
>
```

```

        </div >
    </div >
</div >
{% empty %}
    < p > No courses available.</p >
{% endfor %}
</div >
</div >
{% endblock %}

```

2. **Task:** Create `course_detail.html` in `templates/courses/`.

```

{% extends "courses/base.html" %}
{% block title %}{{ course.title }}{% endblock %}

{% block content %}
< div class = "container" >
    < h1 > {{ course.title }} </h1 >
    < p class = "lead" > {{ course.description }} </p >
    < h2 > Lessons </h2 >
    < ul class = "list - group" >
        {% for lesson in lessons %}
            < li class = "list - group - item" > {{ lesson.title }} </li >
        {% endfor %}
    </ul >
    < a href = "/" class = "btn btn - secondary mt - 3" > Back to Courses </a >
</div >
{% endblock %}

```

3. **Why This Matters:** These templates ensure a visually appealing and responsive UI.

5. Test the Application

1. **Task:** Run the development server.

```
python manage.py runserver
```


2. **Task:** Visit <http://127.0.0.1:8000/> and verify:
 - Courses are displayed in Bootstrap cards.
 - Clicking on a course shows its details.
-

Lab Deliverables

1. Updated views.py and urls.py.
 2. Templates using Bootstrap (base.html, course_list.html, course_detail.html).
 3. Screenshots of:
 - Course list page with Bootstrap styling.
 - Course detail page with Bootstrap elements.
-

Additional Questions for Students

1. How does Bootstrap improve the UI compared to plain HTML?
 2. What are the advantages of using extends in Django templates?
 3. How can you customize Bootstrap styles further in your project?
-

Extension of Lab Work 4: Django Views, Templates, and Local Bootstrap Integration

Objective

By the end of this lab, students will:

1. Understand Django views and templates.
 2. Integrate Bootstrap **without CDN** by using local files.
 3. Create a professional-looking UI for their Django application.
-

Step 1: Download and Add Bootstrap to the Project

1. Download Bootstrap

1. Visit the official Bootstrap website:
[Bootstrap Download](#)
2. Download the latest version.
3. Extract the downloaded files.

2. Add Bootstrap Files to the Django Project

1. Create a static folder inside the courses app:

```
courses/  
  static/  
    courses/  
      bootstrap/  
        css/  
        js/
```

2. Copy the extracted Bootstrap files into the appropriate directories:
 - Place Bootstrap CSS files inside static/courses/bootstrap/css/
 - Place Bootstrap JavaScript files inside static/courses/bootstrap/js/

Step 2: Configure Django to Serve Static Files

1. **Task:** Add static files settings in settings.py:

```
import os
```

```
STATIC_URL = '/static/'
```

```
STATICFILES_DIRS = [os.path.join(BASE_DIR, 'courses/static')]
```

2. **Why This Matters:** This tells Django where to look for static files.

Step 3: Create a Base Template with Bootstrap

1. **Task:** Create a base.html file inside templates/courses/:

```
<!DOCTYPE html >
```

```
<html lang = "en" >
```

```
<head >
```

```
<meta charset = "UTF - 8" >
```

```
<meta name = "viewport" content = "width = device - width, initial - scale = 1" >
```

```
<title > {% block title %}EduLearn{% endblock %} </title >
```

```
{% load static %}
```

```
<!-- Local Bootstrap CSS -->
```

```

< link rel = "stylesheet" type = "text/css" href
      = "{% static 'courses/bootstrap/css/bootstrap.min.css' %}" >
</head >
< body class = "bg - light" >
  < nav class = "navbar navbar - expand - lg navbar - dark bg - primary" >
    < div class = "container" >
      < a class = "navbar - brand" href = "/" > EduLearn </a >
      < button class = "navbar - toggler" type = "button" data - bs - toggle
            = "collapse" data - bs - target = "#navbarNav" >
        < span class = "navbar - toggler - icon" ></span >
      </button >
      < div class = "collapse navbar - collapse" id = "navbarNav" >
        < ul class = "navbar - nav" >
          < li class = "nav - item" >< a class = "nav - link" href = "/" > Courses </a >
          </li >
          < li class = "nav - item" >< a class = "nav - link" href = "/enroll/" > Enroll
          </a ></li >
        </ul >
      </div >
    </div >
  </nav >
  < div class = "container mt - 4" >
    {% block content %}{% endblock %}
  </div >
  < ! - - Local Bootstrap JS - - >
  < script src = "{% static 'courses/bootstrap/js/bootstrap.bundle.min.js' %}" >
    </script >
</body >
</html >

```

2. **Why This Matters:** This template ensures a uniform layout across all pages.

Step 4: Create Views for Course Listing and Details

1. **Task:** Define Django views to display courses.

- Open views.py and add:

```
from django.shortcuts import render, get_object_or_404
from .models import Course
```

```
def course_list(request):
    courses = Course.objects.all()
    return render(request, 'courses/course_list.html', {'courses': courses})
```

```
def course_detail(request, course_id):
    course = get_object_or_404(Course, id = course_id)
    lessons = course.lessons.all()
    return render(request, 'courses
                    /course_detail.html', {'course': course, 'lessons': lessons})
```

2. **Why This Matters:** Django views process data and pass it to templates.
-

Step 5: Update URLs for the Views

1. **Task:** Define URL patterns in urls.py:

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.course_list, name = 'course_list'),
    path('<int:course_id>', views.course_detail, name = 'course_detail'),
]
```

Step 6: Create Templates with Bootstrap

1. **Task:** Create course_list.html inside templates/courses/.

```
{% extends "courses/base.html" %}

{% block title %}Courses{% endblock %}

{% block content %}
```

```

< div class = "container" >

  < h1 class = "text - center" > Available Courses </h1 >

  < div class = "row" >

    {% for course in courses %}

      < div class = "col - md - 4" >

        < div class = "card mb - 4" >

          < div class = "card - body" >

            < h5 class = "card - title" > {{ course.title }} </h5 >

            < p class = "card - text" > {{ course.description }} </p >

            < a href = "{{ course.id }}" class = "btn btn - primary" > View Details </a
              >

          </div >

        </div >

      </div >

    {% empty %}

      < p > No courses available. </p >

    {% endfor %}

  </div >

{% endblock %}

```

2. **Task:** Create course_detail.html inside templates/courses/.

```

{% extends "courses/base.html" %}

{% block title %}{{ course.title }}{% endblock %}

```

```

{% block content %}

  < div class = "container" >

    < h1 > {{ course.title }} </h1 >

    < p class = "lead" > {{ course.description }} </p >

    < h2 > Lessons </h2 >

    < ul class = "list - group" >

      {% for lesson in lessons %}

```

```
< li class = "list - group - item" > {{ lesson.title }} </li >
{% endfor %}
</ul >
< a href = "/" class = "btn btn - secondary mt - 3" > Back to Courses </a >
</div >
{% endblock %}
```

3. **Why This Matters:** These templates ensure a visually appealing and responsive UI using Bootstrap.
-

Step 7: Test the Application

1. **Task:** Run the development server.

python manage.py runserver

2. **Task:** Visit <http://127.0.0.1:8000/> and verify:
 - Courses are displayed in Bootstrap cards.
 - Clicking on a course shows its details.
 - Bootstrap styles load correctly from the local directory.
-

Lab Deliverables

1. Updated views.py and urls.py.
 2. Local Bootstrap files stored in the static/ directory.
 3. Templates using Bootstrap (base.html, course_list.html, course_detail.html).
 4. Screenshots of:
 - Course list page with Bootstrap styling.
 - Course detail page with Bootstrap elements.
-

Additional Questions for Students

1. Why is it beneficial to use local Bootstrap files instead of a CDN?
2. How does Django's `{% load static %}` work in templates?
3. What are the advantages of using a base template (base.html)?